

Scaling, Horovod, and Picking the Right Tool

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

Two Scaling Questions

Strong scaling

Fixed problem, more GPUs.

Question: does my EXISTING job finish faster with 8 GPUs instead of 2?

Ideal: time = $1/N$. Reality: runs into comm floors.

Matters for: eval/hyperparam sweeps, research iteration, fixed paper deadlines.

Weak scaling

Fixed per-GPU work, more GPUs.

Question: if I scale to a BIGGER problem, does wall time stay constant?

Ideal: time is constant. Reality: comm adds overhead.

Matters for: pretraining a larger model, using more data in fixed wall time.

Most real AI workloads are WEAK scaling — we want to train bigger things with our extra compute, not just finish the same job faster.

Where Does the Missing Efficiency Go?

Category	Time spent	Fix
Useful compute	64%	—
All-reduce (not overlapped)	12%	Larger model / smaller buckets / bigger grad accum
Stragglers (slow GPUs)	10%	Homogeneous nodes; pin to healthy GPUs
Load imbalance	6%	Even sharding; same-shape inputs (pad or drop)
Host overhead	4%	Fewer Python callbacks; compiled model
Data loading stalls	4%	pin_memory=True, prefetch, num_workers (Week 4)

MEASURE FIRST: the PyTorch profiler + `torch.cuda.Event` give you the actual breakdown. Don't guess which bar is tallest — profile and attack the biggest one.

Optimisation Levers (In Order of ROI)

- 1 **Mixed precision (AMP)** — fp16/bf16 halves memory and bandwidth; 1.5-2x speedup. Should be on by default.
- 2 **Larger per-GPU batch + gradient accumulation** — Amortises all-reduce across more work — the dominant lever for comm efficiency.
- 3 **Tune NCCL bucket size and count** — Default 25 MB. Large models benefit from bigger buckets; small models from more, smaller buckets.
- 4 **Gradient compression (PowerSGD, signSGD)** — Rare but powerful — 10-100x less traffic, small accuracy hit in most cases.
- 5 **Hierarchical all-reduce (tree algorithms)** — NCCL picks this automatically on clustered topologies. Nothing to tune.
- 6 **Overlap opt-step with next batch's comm** — Advanced. Most gains already captured by DDP's backward hook.

DDP vs FSDP vs DeepSpeed vs Horovod — Quick Comparison

	Memory efficiency	Setup complexity	Speed (fits-in-memory case)	Framework support
DDP	Low (full replica)	Low	Fastest	PyTorch only
FSDP	High (full shard)	Medium	Near-DDP with overlap	PyTorch native
DeepSpeed ZeRO	Highest (+ CPU offload)	Medium-High	Comparable to FSDP	PyTorch (via plugin)
Horovod	Low (full replica)	Medium	Comparable to DDP	PyTorch, TF, MXNet

Horovod — MPI-Based Alternative

What it is

Uber's 2017 distributed training library. Uses MPI (not c10d) for rendezvous and NCCL for collectives. Framework-agnostic: TF, PyTorch, MXNet. Launcher: horovodrun (wraps mpirun). Activity has declined since PyTorch added first-class DDP.

When Horovod still wins

Existing TensorFlow 1.x/2.x codebase.
HPC cluster with Slurm + MPI already set up.
Need a framework-portable training script.
Legacy integration where rewriting would cost more than maintaining.

```
import horovod.torch as hvd
hvd.init()
dist.init_process_group
torch.cuda.set_device(hvd.local_rank())
optim = torch.optim.Adam(model.parameters(), lr=1e-3)
optim = hvd.DistributedOptimizer(optim,
                                # <-- instead of DDP wrap
                                named_parameters=model.named_parameters())
hvd.broadcast_parameters(model.state_dict(), root_rank=0)
hvd.broadcast_optimizer_state(optim, root_rank=0)
# Training loop — identical to DDP.
# Launch: horovodrun -np 8 -H node0:4,node1:4 python train.py
```

Decision Framework — Pick the Right Tool

Scenario	Pick	Why
7B model, 8-32 GPUs, fits in GPU memory	DDP	Simplest. Battle-tested. What 90% of teams use.
13-70B, memory-tight but per-layer fits	FSDP (PyTorch-native)	Shards weights+grads+opt state; keeps DDP's simplicity.
Very tight memory, want library-level optimisation	DeepSpeed ZeRO-3	More knobs than FSDP; CPU offload options.
Existing TF/MXNet codebase, HPC cluster	Horovod	Framework-portable; plays nicely with MPI + Slurm.
One single layer doesn't fit on a GPU	Tensor Parallel (Megatron)	Can't split batch; must split the weights themselves.
Trillion-parameter frontier model	3D Parallel (Megatron-DeepSpeed)	Combine data + tensor + pipeline. GPT-4-class setup.

When in doubt: DDP. When it OOMs(out of memory): FSDP. When those aren't enough: you're at frontier scale.

Scaling Efficiency — A Worked Example

$$\text{eff}(N) = \text{throughput}(N) / (N * \text{throughput}(1))$$

`throughput(1)` = 420 samples/sec (single GPU baseline)

`throughput(8)` = 3,190 samples/sec (8-GPU DDP run)

$\text{eff}(8) = 3190 / (8 * 420) = 3190 / 3360 = 0.949 \rightarrow 94.9\%$ scaling efficiency

Compare against the stacked-bar breakdown from earlier:

64% useful compute + 12% comm + 10% stragglers + ... doesn't map 1:1 to `eff(N)` --

`eff(N)` is measured END-TO-END; the breakdown explains WHERE inside that number the loss lives.

- 94.9% at 8 GPUs is excellent — typical production NVLink+IB clusters land in the 85-95% range for well-tuned DDP jobs
- Below ~80%: profile immediately. Above ~95%: you're likely compute-bound already, further tuning has diminishing returns
- Always report BOTH `throughput(N)` and `eff(N)` — throughput alone hides whether you're wasting GPU-hours

Week 7 Recap

1 Name the four parallelism axes

Data, tensor, pipeline, sharded. Know when each applies.

2 Write and launch a DDP training script

`torchrun`; `sampler.set_epoch`; `save .module.state_dict()`.

3 Explain ring all-reduce

Reduce-scatter + all-gather. $2(N-1)/N$ bandwidth.

4 Debug NCCL problems

`NCCL_DEBUG=INFO`, `NCCL_SOCKET_IFNAME`, rendezvous via telnet.

5 Diagnose scaling inefficiency

Profile; identify comm/imbalance/overhead sources.

6 Pick the right tool for a scenario

DDP vs FSDP vs DeepSpeed vs Horovod vs tensor/pipeline.

Bridge to Week 8

- Week 7 taught you to TRAIN at scale. Week 8 teaches you to SHIP models that survived training.

NEXT WEEK: Week 8 — Scalable Training Dynamics & Deployment Engineering.